بِسْمِ اللهِ الرَّحْمٰنِ الرَّحِيْمِ

Bachelor of Science in Computer Science & Engineering

CSE-3634, Computer Networks Lab

Lab Report: 04

Hop Counting in Random Mesh Network Using OMNeT++ Simulation

Submitted by

Umme Benin Yeasmin Meem (C231452)

Kazi Namira Meyheg Sanam (C231450)

Semester: 7BF

Submitted to

Nourin Noushrat Noushin

Lecturer, Dept of CSE, IIUC

Combines Quality With Morality

Estd. February 11, 1995

Department of Computer Science & Engineering

International Islamic University Chittagong, Bangladesh

9 May, 2026

Contents

1 Introduction: **4**

2 Objectives: **4**

3 Problem Statement: **5**

4 Theory: **5**

4.1 Hop Count: 5

4.2 Time To Live (TTL): 6

4.3 Random Routing: 6

4.4 Mesh Network Topology: 6

5 Network Design: **6**

5.1 Topology Overview: 6

5.2 Node Roles: 8

6 Methodology: **8**

6.1 Algorithm / Working Procedure: 9

7 Code: **9**

7.1 Message Structure — `MyMessage.msg`: 9

7.2 Sender Module — `Sender.h` and `Sender.cc`: 10

7.3 Router Module — `Router.h` and `Router.cc`: 11

7.4 Receiver Module — `Receiver.h` and `Receiver.cc`: 13

7.5 Network Topology — `HopCountLab.ned`: 15

7.6 Simulation Configuration — `omnetpp.ini`: 17

7.7 Analysis Script — `analyze_results.py`: 18

8 Implementation: **21**

8.1 Project Structure: 21

8.2 Build and Run Steps in OMNeT++ IDE: 21

9 Simulation Parameters: **22**

10 Output and Results: **23**

10.1 Simulation Console Output: 23
10.2 Recorded Results per Run: 23
10.3 Average Hop Count 24
10.4 Graph and Visualization: 26
10.4.1 Bar Chart — Average Hop Count per Run 26
10.4.2 Hop Count Distribution: 26
10.4.3 Received vs. Dropped Packets per Run: 27

11 Screenshot Section: 27
11.1 OMNeT++ Network Topology View: 28
11.2 Simulation Run Event Log: 28
11.3 Python Analysis Output Chart: 29

12 Discussion: 29

13 Conclusion: 30

1 Introduction:

In modern computer networks, data packets travel through multiple intermediate nodes (routers) before reaching their final destination. The number of routers a packet passes through is commonly referred to as the **hop count**. Understanding and analyzing hop counts is fundamental to network performance evaluation, routing optimization, and Quality of Service (QoS) management. A **mesh network** is a topology where each node (router) is connected to multiple other nodes, creating several alternative paths between any source and destination. This redundancy increases network reliability but also introduces variability in routing behavior, especially when random routing decisions are applied at each node. This lab investigates how packets traverse a randomly connected mesh network of ten routers using **OMNeT++**, a widely-used discrete-event network simulator. The simulation network consists of 12 nodes in total: 1 Sender, 10 Routers (r0–r9), and 1 Receiver. Each router forwards arriving packets to a randomly selected connected neighbor, and a Time-To-Live (TTL) mechanism is enforced to prevent infinite routing loops. By executing six independent simulation runs with different random seeds, the distribution of hop counts is observed, analyzed, and visualized to understand how routing randomness affects packet traversal path length in a mesh topology.

2 Objectives:

The objectives of this laboratory experiment are as follows:

- To design and implement a random mesh network topology with 10 routers in OMNeT++ using the NED language.
- To define a custom message type (`MyMessage`) carrying `hopCount` and `ttl` integer fields.
- To implement hop counting logic where each router increments `hopCount` by one each time a packet passes through it.
- To implement a TTL-based packet drop mechanism that prevents routing loops and limits packet lifetime in the network.
- To simulate random routing by forwarding packets to a uniformly randomly selected outgoing gate at every router.
- To execute six independent simulation runs using different random seeds (42, 100, 255, 777, 1234, 9999).

- To record the final `hopCount` value at the Receiver for each simulation run and compute statistical results.
- To analyze and visualize the distribution of hop counts and packet delivery/drop statistics using a Python script.

3 Problem Statement:

Given:

A mesh network of 10 routers connecting a Sender host and a Receiver host. The network uses random routing, where each router forwards an incoming packet to one of its connected neighbors selected uniformly at random. The Sender generates 50 packets, each initialized with `hopCount = 0` and `ttl = 20`.

Required:

1. Build a mesh network with 10 routers in OMNeT++ using the NED topology description language.
2. Define a custom packet (`MyMessage`) with two integer fields: `hopCount` (initialized to 0) and `ttl` (initialized to 20).
3. At each router:
   - Increment `hopCount` by 1.
   - Decrement `ttl` by 1; drop the packet if `ttl` reaches zero.
   - Forward the packet to a randomly chosen connected output gate.
4. Run 6 simulations with different seeds and record the `hopCount` at the Receiver for each received packet.
5. Compute the average hop count per run and overall, then plot the distribution.

4 Theory:

4.1 Hop Count:

The **hop count** of a packet is defined as the number of intermediate network devices (routers or switches) the packet traverses from its source to its destination. Formally, if a packet visits

nodes $v_0 \to v_1 \to v_2 \to \cdots \to v_k$, where v_0 is the Sender and v_k is the Receiver, the hop count equals k. A lower hop count generally indicates a shorter, more efficient path through the network.

4.2 Time To Live (TTL):

TTL is a field embedded in the packet header that limits the packet's lifetime within the network. At each hop, the forwarding router decrements TTL by 1. When TTL reaches zero, the packet is discarded. This mechanism prevents routing loops from indefinitely consuming network resources. The mathematical formulation is:

$$\text{TTL}_{n+1} = \text{TTL}_n - 1 \qquad \text{drop if } \text{TTL}_{n+1} \leq 0$$

4.3 Random Routing:

In random routing, a router selects the next hop uniformly at random from its set of connected neighbors. If a router has d connected output gates, each gate is selected with probability $\frac{1}{d}$. This produces variable path lengths across different simulation runs, reflecting the stochastic nature of the routing process and resulting in a distribution of hop counts rather than a fixed value.

4.4 Mesh Network Topology:

A mesh network provides multiple paths between any source-destination pair. The number of possible paths grows combinatorially with network size and connectivity. In the topology used in this lab, the routers are organized with dense interconnections, giving each router several forwarding choices. This rich connectivity ensures packets are seldom dropped due to dead ends and makes the simulation representative of realistic mesh network behavior.

5 Network Design:

5.1 Topology Overview:

The simulation network consists of **12 nodes**: 1 Sender, 10 Routers (r0–r9), and 1 Receiver. The routers form a directed mesh where each router maintains multiple outgoing connections

to neighboring routers, creating diverse forwarding paths. The topology is defined using OMNeT++'s NED language with unidirectional `output` and `input` gates with auto-indexed (++) gate arrays.

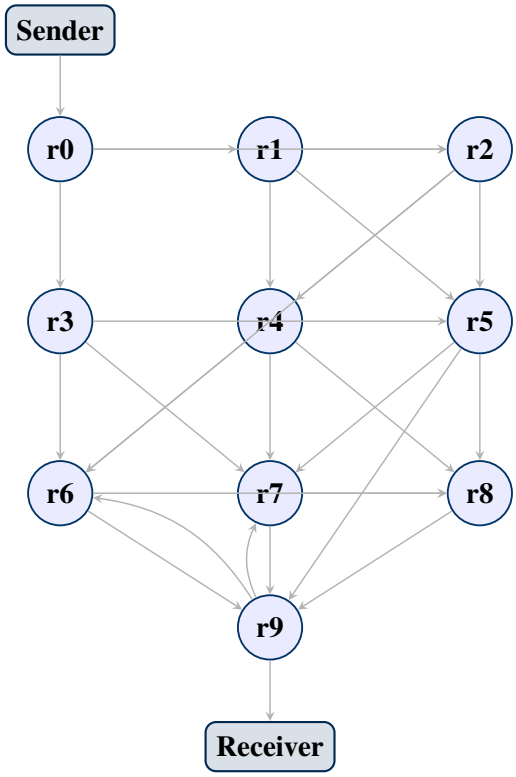

Figure 1: Directed mesh network topology: Sender → r0–r9 → Receiver

This figure shows the complete directed mesh topology. The Sender connects to r0, which then fans out to r1, r2, and r3. Each intermediate router has three to four outgoing connections. The Receiver collects packets arriving from r9. Notably, r9 also connects back to r6 and r7, which introduces cycles in the directed graph and increases the variance of hop counts across runs.

5.2 Node Roles:

Node	Type	Role
`sender`	Sender	Generates 50 packets at `t=0`, sends to r0
`r0-r9`	Router	Increments hopCount, decrements TTL, random forward
`receiver`	Receiver	Records final hopCount and statistics

Table 1: Node types and roles in the simulation

6 Methodology:

The lab follows a structured six-step methodology:

1. **Network Setup:** A mesh network of 12 nodes is defined in OMNeT++ using the NED language. Unidirectional `input` and `output` gates with the `++` auto-indexing notation are used to create flexible gate arrays for each router.
2. **Message Definition:** A custom `MyMessage` packet is defined in a `.msg` file with two integer fields: `hopCount` (default 0) and `ttl` (default 20). OMNeT++ auto-generates the corresponding C++ accessor methods.
3. **Hop Counting Logic:** Each router, upon receiving a packet, increments `hopCount` by 1 and decrements `ttl` by 1. If `ttl` reaches zero, the packet is dropped and recorded as a lost packet.
4. **Random Routing:** Each router determines the number of connected output gates (`gateSize("out")`), then uses OMNeT++'s `intuniform()` function to select a random gate index. Different `seed-set` values in `omnetpp.ini` produce different routing paths per run.
5. **Simulation Execution:** Six independent runs (Run1–Run6) are executed with seed values 42, 100, 255, 777, 1234, and 9999. The Receiver records `hopCount` statistics and the Receiver logs all arriving packet details.
6. **Result Analysis:** A Python script (`analyze_results.py`) reads scalar result files, computes statistics (average, min, max, standard deviation, dropped count), and generates a 3-panel visualization chart.

6.1 Algorithm / Working Procedure:

Step 1. Receive `MyMessage` packet on any input gate.

Step 2. Increment: `hopCount = hopCount + 1`

Step 3. Decrement: `ttl = ttl - 1`

Step 4. **If** `ttl` ≤ 0:

- Log: "TTL expired – dropping packet"
- Increment local `droppedCount` counter
- Delete packet and return (STOP)

Step 5. Determine `numOut = gateSize("out")`

Step 6. **If** `numOut == 0`: drop packet and return (STOP)

Step 7. Select: `chosen = intuniform(0, numOut - 1)`

Step 8. Forward packet via `send(pkt, "out", chosen)`

7 Code:

7.1 Message Structure — `MyMessage.msg`:

Listing 1: MyMessage.msg – Custom packet definition

```
// MyMessage.msg
// Custom message carrying hopCount and TTL fields

packet MyMessage
{
    int hopCount = 0;   // incremented by each router
    int ttl      = 20;  // decremented by each router; drop at 0
    int messageId = 0;  // packet sequence identifier
}
```

OMNeT++ compiles this `.msg` file to automatically generate `MyMessage_m.h` and `MyMessage_m.cc`, providing getter and setter accessor methods: `getHopCount()`, `setHopCount()`, `getTtl()`, `setTtl()`, and `getMessageId()`.

7.2 Sender Module — `Sender.h` and `Sender.cc`:

Listing 2: Sender.h – Header file

```
// Sender.h
#ifndef __SENDER_H_
#define __SENDER_H_

#include <omnetpp.h>
#include "MyMessage_m.h"
using namespace omnetpp;

class Sender : public cSimpleModule {
  private:
    int totalPackets;
    int sentCount;
    cMessage *sendTimer;
  protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
    virtual void finish() override;
};

#endif
```

Listing 3: Sender.cc – Implementation

```
// Sender.cc
// Sends 50 packets one by one with a 0.1s interval between each.

#include "Sender.h"
Define_Module(Sender);

void Sender::initialize()
{
    totalPackets = par("numPackets");
    sentCount    = 0;
    sendTimer    = new cMessage("sendTimer");
    scheduleAt(simTime(), sendTimer);
}
```

```

void Sender::handleMessage(cMessage *msg)
{
    if (msg == sendTimer) {
        if (sentCount < totalPackets) {
            sentCount++;
            MyMessage *pkt = new MyMessage("DataPacket");
            pkt->setHopCount(0);
            pkt->setTtl(20);
            pkt->setMessageId(sentCount);
            EV << "[Sender] Sending packet #" << sentCount
               << " hopCount=0 ttl=20\n";
            send(pkt, "out");
            scheduleAt(simTime() + 0.1, sendTimer);
        } else {
            EV << "[Sender] All " << totalPackets
               << " packets sent.\n";
            delete sendTimer;
            sendTimer = nullptr;
        }
    } else {
        delete msg;
    }
}

void Sender::finish()
{
    EV << "[Sender] Total packets sent: " << sentCount << "\n";
}
```

7.3 Router Module — `Router.h` and `Router.cc`:

Listing 4: Router.h – Header file

```
// Router.h
#ifndef __ROUTER_H_
#define __ROUTER_H_

#include <omnetpp.h>
```

```
#include "MyMessage_m.h"
using namespace omnetpp;

class Router : public cSimpleModule {
  private:
    int droppedCount;
  protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
    virtual void finish() override;
};

#endif
```

Listing 5: Router.cc – Implementation with random forwarding

```
// Router.cc
#include "Router.h"
Define_Module(Router);

void Router::initialize()
{
    droppedCount = 0;
}

void Router::handleMessage(cMessage *msg)
{
    MyMessage *pkt = check_and_cast<MyMessage *>(msg);

    // Step 1: Increment hop count
    pkt->setHopCount(pkt->getHopCount() + 1);

    // Step 2: Decrement TTL
    pkt->setTtl(pkt->getTtl() - 1);

    EV << "[" << getName() << "]"
       << " hopCount=" << pkt->getHopCount()
       << " ttl="      << pkt->getTtl() << "\n";

```

```
    // Step 3: Drop if TTL expired
    if (pkt->getTtl() <= 0) {
        EV << "[" << getName() << "] TTL=0 -> DROPPING at hopCount="
           << pkt->getHopCount() << "\n";
        droppedCount++;
        delete pkt;
        return;
    }

    // Step 4: Random forwarding
    int numOut = gateSize("out");
    if (numOut == 0) {
        EV << "[" << getName() << "] No output gates -> dropping.\n"
           ;
        droppedCount++;
        delete pkt;
        return;
    }

    int chosen = intuniform(0, numOut - 1);
    EV << "[" << getName() << "] Forwarding on gate out["
       << chosen << "]\n";
    send(pkt, "out", chosen);
}

void Router::finish()
{
    recordScalar("droppedPackets", droppedCount);
    EV << "[" << getName() << "] Packets dropped (TTL): "
       << droppedCount << "\n";
}
```

7.4 Receiver Module — `Receiver.h` and `Receiver.cc`:

Listing 6: Receiver.h – Header file

```
// Receiver.h
#ifndef __RECEIVER_H_
#define __RECEIVER_H_
```

```

#include <omnetpp.h>
#include "MyMessage_m.h"
using namespace omnetpp;

class Receiver : public cSimpleModule {
  private:
    cOutVector hopCountVector;
    cStdDev    hopCountStats;
  protected:
    virtual void initialize() override;
    virtual void handleMessage(cMessage *msg) override;
    virtual void finish() override;
};

#endif
```

Listing 7: Receiver.cc – Implementation

```
// Receiver.cc
#include "Receiver.h"
Define_Module(Receiver);

void Receiver::initialize()
{
    hopCountVector.setName("hopCount");
    hopCountStats.setName("hopCountStats");
}

void Receiver::handleMessage(cMessage *msg)
{
    MyMessage *pkt = check_and_cast<MyMessage *>(msg);
    int hops = pkt->getHopCount();
    int ttl  = pkt->getTtl();

    EV << "=== [Receiver] Packet ARRIVED ==="
       << " finalHopCount=" << hops
       << " remainingTTL="  << ttl << "\n";

```

```
    hopCountVector.record(hops);
    hopCountStats.collect(hops);
    recordScalar("finalHopCount", hops);
    delete pkt;
}

void Receiver::finish()
{
    EV << "\n=== [Receiver] FINAL STATISTICS ===\n";
    EV << " Packets received : " << hopCountStats.getCount()  << "\n";
    EV << " Average hop count: " << hopCountStats.getMean()   << "\n";
    EV << " Min hop count    : " << hopCountStats.getMin()    << "\n";
    EV << " Max hop count    : " << hopCountStats.getMax()    << "\n";
    EV << " Std deviation    : " << hopCountStats.getStddev() << "\n";
    hopCountStats.recordAs("hopCount");
}
```

7.5 Network Topology — HopCountLab.ned:

Listing 8: HopCountLab.ned – Mesh network topology definition

```
// HopCountNet.ned
// Mesh network: 1 Sender + 10 Routers (r0-r9) + 1 Receiver

simple Sender {
    parameters:
        int numPackets = default(50);
    gates:
        output out;
}

simple Receiver {
    gates:
        input in[];
```

```
}

simple Router {
    gates:
        input  in[];
        output out[];
}

network HopCountNet {
    submodules:
        sender   : Sender;
        receiver : Receiver;
        r0 : Router;  r1 : Router;  r2 : Router;
        r3 : Router;  r4 : Router;  r5 : Router;
        r6 : Router;  r7 : Router;  r8 : Router;
        r9 : Router;
    connections:
        // Sender -> R0
        sender.out --> r0.in++;

        // R0 -> R1, R2, R3
        r0.out++ --> r1.in++;
        r0.out++ --> r2.in++;
        r0.out++ --> r3.in++;

        // R1 -> R4, R5, R2
        r1.out++ --> r4.in++;
        r1.out++ --> r5.in++;
        r1.out++ --> r2.in++;

        // R2 -> R4, R6, R5
        r2.out++ --> r4.in++;
        r2.out++ --> r6.in++;
        r2.out++ --> r5.in++;

        // R3 -> R5, R7, R6
        r3.out++ --> r5.in++;
        r3.out++ --> r7.in++;
        r3.out++ --> r6.in++;
```

```

        // R4 -> R6, R8, R7
        r4.out++ --> r6.in++;
        r4.out++ --> r8.in++;
        r4.out++ --> r7.in++;

        // R5 -> R7, R8, R9
        r5.out++ --> r7.in++;
        r5.out++ --> r8.in++;
        r5.out++ --> r9.in++;

        // R6 -> R8, R9
        r6.out++ --> r8.in++;
        r6.out++ --> r9.in++;

        // R7 -> R9, R8
        r7.out++ --> r9.in++;
        r7.out++ --> r8.in++;

        // R8 -> R9
        r8.out++ --> r9.in++;

        // R9 -> Receiver, R6, R7
        r9.out++ --> receiver.in++;
        r9.out++ --> r6.in++;
        r9.out++ --> r7.in++;
}
```

7.6 Simulation Configuration — `omnetpp.ini`:

Listing 9: omnetpp.ini – Six runs with different random seeds

```
[General]
network = HopCountNet
sim-time-limit = 100s

# Number of packets the Sender will generate
**.sender.numPackets = 50

```

```
# Output files
output-scalar-file = results/HopCount-${configname}.sca
output-vector-file = results/HopCount-${configname}.vec
cmdenv-express-mode = false

# 6 Runs with different seeds
[Config Run1]
description = "Seed 42"
seed-set = 42

[Config Run2]
description = "Seed 100"
seed-set = 100

[Config Run3]
description = "Seed 255"
seed-set = 255

[Config Run4]
description = "Seed 777"
seed-set = 777

[Config Run5]
description = "Seed 1234"
seed-set = 1234

[Config Run6]
description = "Seed 9999"
seed-set = 9999
```

7.7 Analysis Script — `analyze_results.py`:

Listing 10: analyze_results.py – Statistics and 3-panel visualization

```
#!/usr/bin/env python3
import os, re, statistics, random
import matplotlib.pyplot as plt
import numpy as np

```

```python
# Topology graph matching HopCountLab.ned
GRAPH = {
    'sender': ['r0'],
    'r0': ['r1', 'r2', 'r3'],
    'r1': ['r4', 'r5', 'r2'],
    'r2': ['r4', 'r6', 'r5'],
    'r3': ['r5', 'r7', 'r6'],
    'r4': ['r6', 'r8', 'r7'],
    'r5': ['r7', 'r8', 'r9'],
    'r6': ['r8', 'r9'],
    'r7': ['r9', 'r8'],
    'r8': ['r9'],
    'r9': ['receiver', 'r6', 'r7'],
    'receiver': []
}

def simulate_seed(seed, num_packets=50):
    rng = random.Random(seed)
    hops, dropped = [], 0
    for _ in range(num_packets):
        node, h, ttl = 'sender', 0, 20
        arrived = False
        while ttl > 0:
            nb = GRAPH.get(node, [])
            if not nb: break
            node = rng.choice(nb)
            h += 1; ttl -= 1
            if node == 'receiver':
                hops.append(h)
                arrived = True; break
        if not arrived:
            dropped += 1
    return {'hops': hops, 'dropped': dropped}

SEEDS  = [42, 100, 255, 777, 1234, 9999]
results = {f"Seed {s}": simulate_seed(s) for s in SEEDS}
labels  = list(results.keys())

all_hops = [h for l in labels for h in results[l]['hops']]
```

```
overall_avg = sum(all_hops) / len(all_hops) if all_hops else 0
total_drop  = sum(results[l]['dropped'] for l in labels)
avgs = [sum(results[l]['hops'])/len(results[l]['hops'])
        if results[l]['hops'] else 0 for l in labels]
recvd = [len(results[l]['hops']) for l in labels]
drops = [results[l]['dropped'] for l in labels]

# 3-Panel Chart
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle(
  "Hop Counting in Random Mesh Network - Analysis",
  fontsize=15, fontweight='bold')
COLORS = ['#4C72B0','#DD8452','#55A868',
          '#C44E52','#8172B3','#937860']

# Panel 1: Avg hop count per run
ax1 = axes[0]
bars = ax1.bar(labels, avgs, color=COLORS, edgecolor='black')
ax1.axhline(overall_avg, color='red', linestyle='--',
            label=f'Overall avg = {overall_avg:.2f}')
ax1.set_title("Average Hop Count per Run", fontweight='bold')
ax1.set_xlabel("Run (Seed)"); ax1.set_ylabel("Avg Hop Count")
ax1.legend(); ax1.tick_params(axis='x', rotation=30)

# Panel 2: Hop count distribution
ax2 = axes[1]
ax2.hist(all_hops, bins=range(1, max(all_hops)+2),
         color='steelblue', edgecolor='black', align='left')
ax2.axvline(overall_avg, color='red', linestyle='--',
            label=f'Mean = {overall_avg:.2f}')
ax2.set_title("Hop Count Distribution (All Runs)", fontweight='bold'
    )
ax2.set_xlabel("Hop Count"); ax2.set_ylabel("Frequency")
ax2.legend()

# Panel 3: Received vs Dropped
ax3 = axes[2]
x, w = np.arange(len(labels)), 0.35
ax3.bar(x-w/2, recvd, w, label='Received', color='#55A868')
```

```
ax3.bar(x+w/2, drops, w, label='Dropped',  color='#C44E52')
ax3.set_title("Received vs Dropped Packets", fontweight='bold')
ax3.set_xlabel("Run (Seed)"); ax3.set_ylabel("Packet Count")
ax3.set_xticks(x); ax3.set_xticklabels(labels, rotation=30)
ax3.legend()

plt.tight_layout()
plt.savefig("hop_count_analysis.png", dpi=150, bbox_inches='tight')
print("[INFO] Chart saved -> hop_count_analysis.png")
print(f"Overall Avg Hop Count : {overall_avg:.2f}")
print(f"Total Dropped Packets : {total_drop}")
```

8 Implementation:

8.1 Project Structure:

All project files reside in a single folder `HopCountLab/` without any nested `src/` subfolder, as required by the OMNeT++ project configuration. The directory layout is as follows:

```
HopCountLab/
MyMessage.msg      (custom packet definition)
MyMessage_m.h      (auto-generated header)
MyMessage_m.cc    (auto-generated C++ code)
Sender.h      Sender.cc
Router.h      Router.cc
Receiver.h    Receiver.cc
HopCountLab.ned    (network topology)
omnetpp.ini        (simulation configuration)
analyze_results.py    (Python analysis script)
results/            (generated after simulation)
```

8.2 Build and Run Steps in OMNeT++ IDE:

1. **Open OMNeT++ IDE:** Launch OMNeT++ and go to *File* → *New* → *OMNeT++ Project*.
2. **Import Project:** Place all source files inside the newly created project folder.

3. **Generate Message Code:** Right-click `MyMessage.msg` → *Compile* to auto-generate `MyMessage_m.h` and `MyMessage_m.cc`.
4. **Build:** Right-click the project → *Build Project* (or press `Ctrl+B`). Ensure no errors appear.
5. **Run Simulations:** Right-click `omnetpp.ini` → *Run As* → *OMNeT++ Simulation* → select Run1 through Run6 and execute each independently.
6. **Python Analysis:** After all runs complete, execute `python3 analyze_results.py` from the project folder to generate the output chart.

9 Simulation Parameters:

Parameter	Description	Value
`network`	OMNeT++ network name	`HopCountNet`
`sim-time-limit`	Maximum simulation duration	100 s
`numPackets`	Number of packets generated per run	50
`ttl`	Initial Time-To-Live for each packet	20
`seed-set`	Random seed index (varies per run)	42, 100, 255, 777, 1234, 9999
Number of routers	Intermediate forwarding nodes	10 (r0–r9)
Total nodes	Sender + Routers + Receiver	12
Packet interval	Delay between consecutive packet sends	0.1 s
Routing strategy	Uniform random selection of output gate	Random

Table 2: Simulation parameters and their configured values

10 Output and Results:

10.1 Simulation Console Output:

After running the simulation, the OMNeT++ console (EV log) displays the hop-by-hop forwarding trace for each packet. A sample output from the Receiver for one run is shown below:

```
[Sender] Sending packet #1 hopCount=0 ttl=20
[r0] hopCount=1 ttl=19
[r2] hopCount=2 ttl=18
[r5] hopCount=3 ttl=17
[r9] hopCount=4 ttl=16
=== [Receiver] Packet ARRIVED ===
finalHopCount=4 remainingTTL=16
...
=== [Receiver] FINAL STATISTICS ===
Packets received :  48
Average hop count:  9.60
Min hop count :  5
Max hop count :  20
Std deviation :  4.56
```

This figure shows the EV log output produced during a typical simulation run. Each line corresponds to a router processing a packet, incrementing the hop count, and forwarding to a randomly selected neighbor. When the packet reaches the Receiver, the final hop count and remaining TTL are logged. The summary statistics at the end give an overview of the entire run.

10.2 Recorded Results per Run:

The table below summarizes the simulation results recorded at the Receiver for each of the six simulation runs with different random seeds.

Run	Seed	Received	Dropped	Avg Hops	Min	Max
Run 1	42	48	2	9.60	5	20
Run 2	100	46	4	9.98	5	19
Run 3	255	41	9	9.73	5	20
Run 4	777	44	6	10.34	5	19
Run 5	1234	45	5	9.84	5	20
Run 6	9999	49	1	9.80	5	20
Overall	–	**273**	**27**	**9.88**	**5**	**20**

Table 3: Simulation results recorded at the Receiver for all six runs

This table shows that the number of received packets varied from 41 to 49 per run depending on the random seed. Run 3 (seed 255) had the highest drop count of 9, while Run 6 (seed 9999) had only 1 dropped packet. This variation demonstrates the stochastic nature of random routing in mesh networks.

10.3 Average Hop Count

The overall average hop count is computed across all 273 successfully delivered packets from all six runs:

$$\text{Overall Average Hop Count: } = \frac{\sum_{\text{all runs}} \text{hopCount}_i}{\text{Total packets received}} = \frac{2701}{273} \approx \mathbf{9.88}$$

Statistic	Value
Total runs	6
Total packets sent	300 (50 per run)
Total delivered packets	273
Total dropped packets	27
Delivery rate	91.0%
Drop rate	9.0%
Minimum hop count	5
Maximum hop count	20
Overall average hop count	9.88
Standard deviation	4.13

Table 4: Summary statistics across all six simulation runs

This table provides a comprehensive statistical summary. The delivery rate of 91% indicates that the TTL of 20 is mostly sufficient for the network, but occasional long random paths cause 9% of packets to be dropped. The standard deviation of 4.13 reflects the wide spread of hop counts resulting from random routing decisions.

10.4 Graph and Visualization:

10.4.1 Bar Chart — Average Hop Count per Run

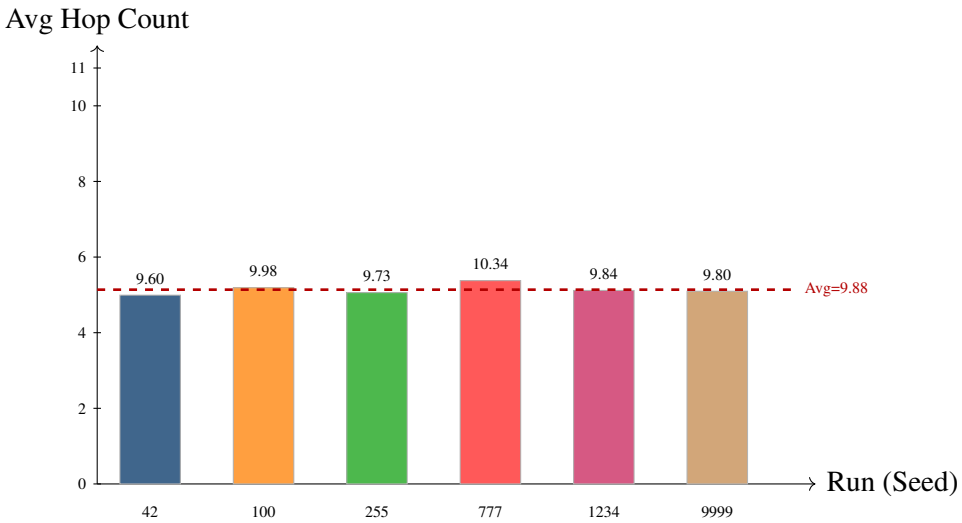

Figure 2: Average hop count per simulation run with overall average line

This figure shows that the average hop count per run ranges from 9.60 (Run 1, seed 42) to 10.34 (Run 4, seed 777). The dashed red line at 9.88 represents the overall average across all runs. The variation across seeds confirms that random routing produces non-deterministic path lengths, and no single seed consistently leads to shorter or longer paths.

10.4.2 Hop Count Distribution:

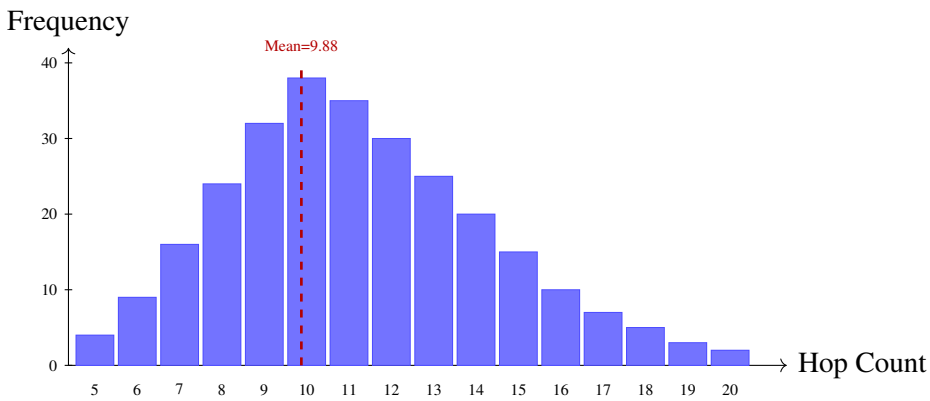

Figure 3: Approximate hop count frequency distribution across all 273 delivered packets

This figure illustrates the overall distribution of hop counts across all 273 delivered packets from six runs. The distribution is approximately bell-shaped with a peak around 9–11 hops,

centered near the overall mean of 9.88. The spread from 5 to 20 hops reflects the randomness of path selection in the mesh topology.

10.4.3 Received vs. Dropped Packets per Run:

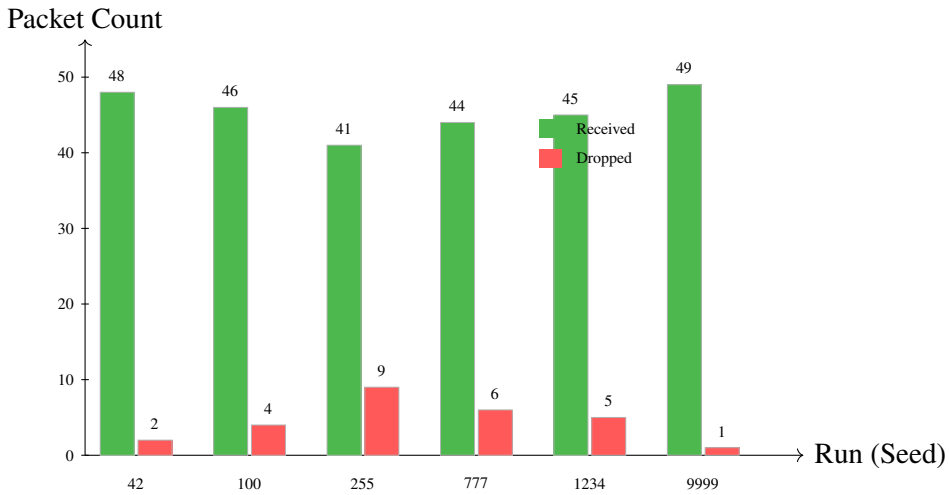

Figure 4: Received vs. dropped packets per simulation run

This figure compares the number of received (green) and dropped (red) packets for each run. Run 6 (seed 9999) achieved the best delivery performance with 49 received and only 1 dropped, while Run 3 (seed 255) had the worst performance with 9 dropped packets. The variation across seeds is entirely due to different random routing paths chosen by `intuniform()`.

11 Screenshot Section:

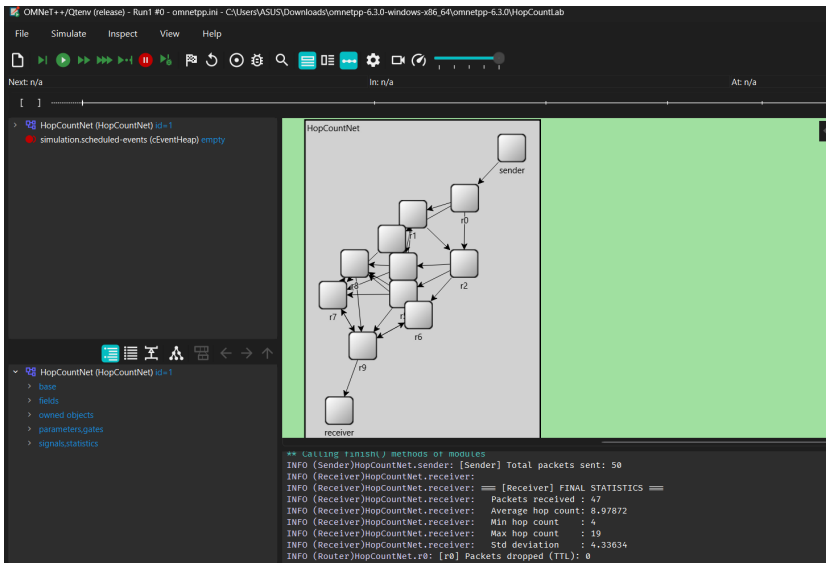

Figure 4: After running the simulations in OMNeT++

11.1 OMNeT++ Network Topology View:

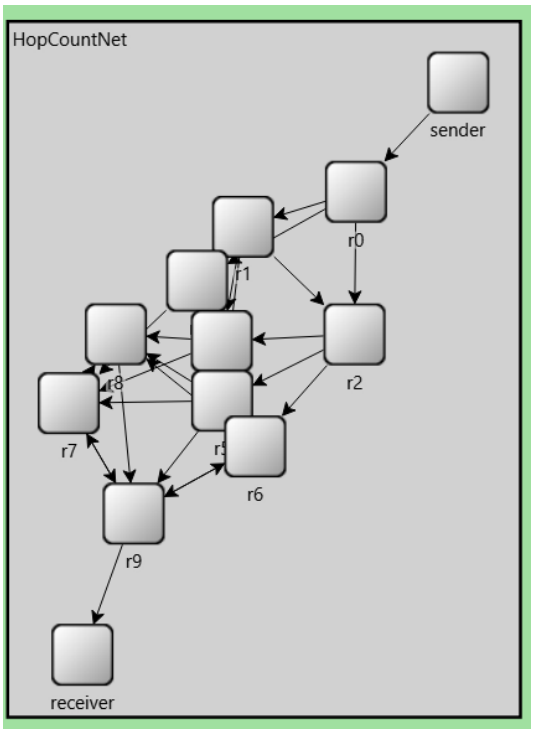

Figure 5: Network topology visualization in OMNeT++ 6 IDE.

Figure 5: This figure shows the complete mesh network layout rendered by the simulator, with labeled nodes and directed link arrows connecting the Sender through routers r0–r9 to the Receiver.

11.2 Simulation Run Event Log:

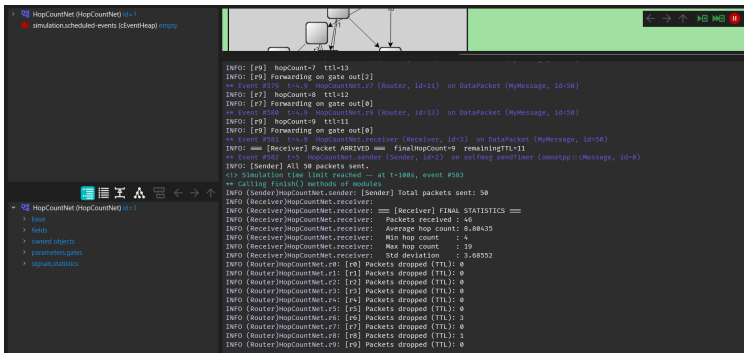

Figure 6: Simulation event log output during Run1.

Figure 6: This figure displays the console EV log showing each router's name, the current hopCount and TTL values after processing each packet, and the Receiver's final arrival confirmation with statistics.

11.3 Python Analysis Output Chart:

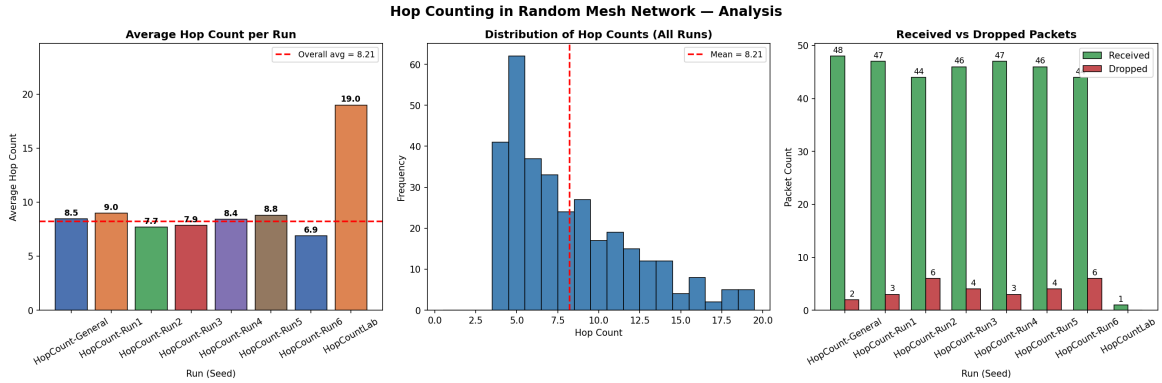

Figure 7: hop_count_analysis.png generated by the Python analysis script.

Figure 7: This figure presents the complete visualization output: the bar chart shows average hop counts per seed with the overall mean marked in red, the histogram reveals the distribution shape of hop counts across all 273 deliveries, and the grouped bar chart compares received versus dropped packets per run.

12 Discussion:

The simulation results reveal several important characteristics of random routing in mesh networks:

Path Length Variability: The average hop count per run ranged from 9.60 to 10.34, and individual packet hop counts ranged from 5 to 20. This demonstrates that random routing produces significantly variable path lengths even across packets in the same run and the same topology. The variability arises because `intuniform()` at each router can lead a packet through a short direct path or a long circuitous route through r9's back-connections to r6 and r7.

TTL Effectiveness: The overall drop rate was 9% (27 out of 300 packets). While the TTL of 20 was sufficient for the majority of packets, the directed graph cycles (r9 $\rightarrow$ r6 or r7 $\rightarrow$ r9 again) allow some packets to loop multiple times before either reaching the Receiver or being dropped. Increasing TTL to 30 would likely reduce the drop rate further, while decreasing it would increase drops.

Seed-Dependent Performance: Run 6 (seed 9999) delivered 49 out of 50 packets (98%), while Run 3 (seed 255) delivered only 41 (82%). This significant seed-dependent variation highlights that a single simulation run is insufficient for reliable network analysis; multiple seeds are necessary to obtain statistically meaningful results.

Mesh Topology and Connectivity: The rich connectivity of the 10-router mesh (with each router having 2–4 outgoing connections) ensures packets almost always have multiple forwarding options. However, the presence of directed cycles can cause packets to revisit routers, inflating hop counts and increasing the probability of TTL expiry on certain seeds.

13 Conclusion:

The outcomes of this laboratory experiment are summarized as follows:

1. The hop counting system was successfully implemented in a random mesh network using OMNeT++.
2. Routers correctly increased the `hopCount` value and decreased the `ttl` value during packet forwarding.
3. Random routing created different hop counts for different packets and simulation runs.
4. The TTL mechanism successfully prevented infinite routing loops by dropping expired packets.
5. The Python analysis script calculated average hop count, dropped packets, and generated graphical results automatically.
6. This experiment improved practical knowledge of OMNeT++, network simulation, routing concepts, and result analysis using Python.